

Atividade Prática 1 — Classificador Bayesiano

Etapa 1 — Relatório de Modelagem do Problema

Autor	Marcus Vinícius Santos de Almeida
Disciplina	Mineração de Dados
Data de entrega	04/09/2026
Domínio escolhido	Predição de Defeitos de Software a partir de métricas estáticas de código
Decisão binária	O arquivo de código-fonte apresentará um defeito reportado? (SIM / NÃO)

1. Descrição do Domínio

1.1 O problema de negócio

Em projetos de software de médio e grande porte, rodar a suíte de testes completa a cada alteração e submeter todo o código a revisão manual detalhada não escala: consome tempo de máquina e, principalmente, tempo de pessoas. Uma estratégia comum é **priorizar** — concentrar esforço de teste e de revisão nos trechos de código com maior probabilidade de conter defeitos.

Este trabalho modela exatamente essa priorização como um problema de classificação binária: dado um arquivo de código-fonte descrito por um conjunto de métricas estáticas e de histórico, decidir se ele deve ser tratado como **propenso a defeito** (SIM) ou **não propenso** (NÃO). A saída serve como sinal de priorização, não como veredito — um arquivo classificado como SIM entra na frente da fila de revisão e ganha atenção extra de testes.

1.2 Por que este domínio

- **Literatura acadêmica consolidada.** Predição de defeitos a partir de métricas de código é uma sub-área madura de engenharia de software empírica, com décadas de estudos, o que dá base sólida para justificar cada feature e, sobretudo, para a análise crítica exigida na Etapa 4.
- **Dados fáceis de obter.** Todas as seis features podem ser extraídas de um repositório Git com ferramentas leves e gratuitas (ou com a biblioteca padrão do Python), sem instrumentação especial nem acesso a bancos de bugs proprietários.
- **Utilidade prática real.** O resultado tem uso imediato: orientar onde gastar o orçamento limitado de teste e revisão.

1.3 O que conta como "defeito reportado"

Adota-se a convenção usual da área: um arquivo recebe rótulo SIM quando, em uma janela de observação definida (por exemplo, a versão seguinte do sistema ou os N meses seguintes), ele foi modificado por um commit de correção — um commit associado a um item de bug em rastreador de issues, ou identificável como correção pela mensagem ("fix", "bug", "corrige", referência a [issue](#)). Arquivos não tocados por nenhum commit de correção nessa janela recebem NÃO. Como os dados de treinamento desta atividade são **sintéticos** (ver Seção 6 e Etapa 2), o rótulo é gerado junto com as features de forma coerente com os padrões descritos na Seção 3.4; a definição acima documenta como o rótulo seria obtido em um cenário real.

2. Unidade de Análise

A "unidade de análise" (o que o enunciado chama de *módulo*) é o **arquivo de código-fonte**.

O estudo de referência para o comportamento de tamanho vs. defeito (Koru et al., 2008) usa a **classe** como módulo. Aqui adota-se o arquivo como simplificação deliberada, pelos seguintes motivos:

1. **Nenhuma das seis features atuais depende do conceito de classe.** As duas features que dependiam — profundidade na árvore de herança (DIT) e acoplamento entre objetos (CBO) —

foram removidas do modelo (ver Seção 3.2 e a fundamentação na Seção 4). O que sobrou (complexidade, LOC, autores, churn, imports, cobertura) é definível para qualquer arquivo de texto de código, independente de o projeto ser orientado a objetos.

2. **Arquivo é unidade universal.** Funciona para Python, para C, para SQL, para qualquer linguagem, e é a granularidade nativa do Git — o que simplifica a coleta de autores e churn.

3. **Evita ferramenta adicional.** Trabalhar no nível de classe exigiria um parser semântico por linguagem para delimitar cada classe e atribuir métricas a ela; no nível de arquivo, as mesmas ferramentas leves bastam.

Declaração de simplificação: ao usar arquivo em vez de classe, agregamos em um só registro elementos que um estudo no nível de classe separaria (um arquivo pode conter várias classes ou funções). Isso torna as métricas por registro um pouco mais "grossas" e pode diluir sinais localizados em uma classe específica. Assume-se esse custo em troca de simplicidade e universalidade da coleta.

3. Modelagem do Problema

3.1 (a) Rótulo alvo

O arquivo apresentará um defeito reportado na janela de observação?

- Valores: **SIM** (propenso a defeito) e **NÃO** (não propenso).
- É a variável que o classificador Naive Bayes vai estimar, calculando $P(\text{SIM} \mid \text{features})$ e $P(\text{NÃO} \mid \text{features})$.
- Critério de rótulo: presença de commit de correção tocando o arquivo na janela (ver Seção 1.3).

3.2 (b) As seis features do domínio

As seis features abaixo são exatamente as definidas no projeto — nenhuma foi inventada ou removida nesta etapa. O conjunto atende ao mínimo exigido (6 a 8). DIT e acoplamento (CBO) foram descartados antes desta etapa por dependerem do conceito de classe e por apresentarem forte correlação com tamanho na literatura recente (Seção 4), o que reduziria a diversidade real de informação sem agregar sinal novo.

#	Feature	Como é coletada	Fundamentação e ponto de atenção
1	Complexidade ciclomática	Radon (radon cc)	McCabe (1976) criou a métrica para apoiar o desenho de casos de teste, não para prever defeitos diretamente. Correlaciona-se fortemente com LOC (evidência empírica de relação linear estável, $R^2 \approx 0,90$ e acima); Shepperd (1988) argumenta que, para boa parte do software, é apenas um <i>proxy</i> para LOC. É o exemplo central de violação da hipótese de independência do Naive Bayes — discussão teórica completa na Seção 5.
2	Linhas de código (LOC)	Radon (radon raw)	Koru et al. (2008), <i>Theory of Relative Defect Proneness</i> : a relação tamanho→defeito segue uma lei de potência com expoente < 1 , ou seja, módulos menores são proporcionalmente mais propensos a defeito (resultado contraintuitivo). "Tamanho" no estudo é literalmente LOC. A discretização (Seção 3.3) precisa refletir essa não-linearidade — não assumir "mais linhas = mais risco" de forma linear.
3	Número de autores distintos	Histórico do Git (PyDriller / git log)	Matsumoto et al. (2010): métricas de desenvolvedor melhoram a predição de defeitos. Weyuker, Ostrand & Bell (2008), " <i>Do Too Many Cooks Spoil the Broth?</i> ": quanto mais pessoas diferentes editam um arquivo, maior tende a ser a propensão a defeito — falta de propriedade clara do código.
4	Churn relativo (% das linhas do arquivo alteradas nos últimos 90 dias)	Histórico do Git (PyDriller)	Nagappan & Ball (2005): usar churn relativo ao tamanho do módulo, não o churn absoluto — churn absoluto é preditor fraco e correlaciona com tamanho; a versão relativa foi desenhada precisamente para reduzir essa dependência.
5	Número de imports / dependências externas	Módulo <code>ast</code> da biblioteca padrão do Python (não é ferramenta externa)	Substitui DIT/CBO. Mede acoplamento de forma leve, sem depender de classe: quantos módulos/pacotes distintos o arquivo importa. Um arquivo com muitas dependências tem maior superfície para quebrar quando qualquer uma delas muda.
6	Cobertura de testes	Coverage.py	Evidência contestada . Vários estudos — inclusive um survey com 235 respostas de sete organizações (Gren & Antinyan, 2017) e um estudo empírico controlado (Inozemtseva & Holmes, 2014) — encontram correlação nula ou fraca entre cobertura e (ausência de) defeitos. Não tratar "mais cobertura → menos defeito" como relação óbvia; é uma limitação assumida do modelo, declarada explicitamente e retomada na análise crítica da Etapa 4.

3.3 (c) Discretização de cada feature

O Naive Bayes desta atividade trabalha com features **categóricas**. As faixas abaixo são a discretização adotada no projeto. Os pontos de corte são razoáveis para código Python de projeto real; o que precisa se manter é **a lógica das faixas**, não a calibração fina de cada número.

Feature	Faixas (baixo → alto)	Observação sobre o significado
Complexidade ciclomática (soma no arquivo)	baixa ≤ 10 · media 11-20 · alta > 20	Bandas clássicas de McCabe (≤ 10 = baixo risco; acima de 20 = difícil de testar). Em código real "alta" concentra risco mas também parte de sinal redundante com LOC (Seção 5); nos dados sintéticos deste projeto essa redundância foi deliberadamente removida (Etapa 2).
LOC (linhas de código)	curto < 50 · medio 50-200 · longo > 200	Curto ≠ seguro. Pela lei de potência de Koru et al. (2008), arquivos curtos têm risco proporcionalmente elevado . Relação em U: a faixa medio tende a ser a mais "tranquila"; curto e longo carregam risco por motivos diferentes (curto = alta densidade proporcional de defeito; longo = volume absoluto de código e de caminhos).
Número de autores distintos	um = 1 · poucos 2-3 · muitos ≥ 4	"Muitos cozinheiros": diluição de responsabilidade e mistura de estilos; um autor indica propriedade clara do código.
Churn relativo = % das linhas do arquivo alteradas nos últimos 90 dias	baixo $< 10\%$ · medio 10-30% · alto $> 30\%$	Medida relativa ao tamanho do arquivo (Nagappan & Ball, 2005), não contagem absoluta de linhas. alto = arquivo que muda muito para o tamanho que tem (área instável / requisito volátil).
Número de imports / dependências externas	poucos 0-3 · medio 4-8 · muitos > 8	muitos = grande superfície de dependência: muitos pontos por onde uma mudança externa pode introduzir defeito.
Cobertura de testes	baixa $< 50\%$ · media 50-80% · alta $> 80\%$	Feature de sinal fraco/contestado — incluída para ser analisada criticamente, não porque se espera forte poder preditivo.

Notas sobre a discretização:

- **LOC e complexidade ciclomática não são discretizadas assumindo relação linear simples com risco.** Para LOC isso é explícito nas três faixas: a intuição de risco é em formato de U, seguindo Koru et al. (2008). Para complexidade, usam-se as bandas de McCabe.
- **Churn é sempre relativo ao tamanho** (percentual do arquivo alterado), nunca a contagem absoluta de linhas ou de commits — decisão apoiada em Nagappan & Ball (2005).
- Todas as features têm **exatamente três categorias**. Isso mantém as tabelas de verossimilhança pequenas e explicáveis linha a linha na arguição, e reduz o número de células com contagem zero (que exigiriam suavização de Laplace mais agressiva).
- Cada corte é um número que o autor consegue justificar oralmente; nenhum é fruto de otimização automática.

3.4 (d) Lógica intuitiva dos padrões de risco

Em linguagem simples, por que a combinação dessas seis features aponta para **maior** ou **menor** propensão a defeito:

Sinais que puxam para SIM (mais propenso a defeito):

- **Código muito ramificado** (complexidade **alta**): mais caminhos de execução, mais condições, mais lugares onde a lógica pode estar errada — e mais difícil de cobrir inteiramente com testes.
- **Muitas mãos no mesmo arquivo** (autores **muitos**): quando ninguém é "dono" do arquivo, convenções se misturam, suposições de um autor são quebradas por outro e detalhes se perdem entre quem entra e quem sai.
- **Arquivo que muda o tempo todo para o tamanho que tem** (churn relativo **alto**): indica área de requisito instável ou design que ainda não assentou; cada alteração é uma oportunidade de introduzir defeito.
- **Arquivo pequeno, porém denso** (LOC **curto** + complexidade **media/alta**): o caso contraintuitivo de Koru et al. — muita lógica comprimida em pouco espaço, frequentemente utilitários centrais chamados de todo lado, com alta taxa de defeito por linha. Na modelagem deste proje-

to, esse padrão aparece como a **soma** dos dois efeitos individuais (LOC baixo + complexidade alta), não como um bônus de interação plantado — ver Seção 5 e a Etapa 4.

- **Muitas dependências externas** (imports **muitos**): amplia a superfície de quebra — uma mudança em qualquer biblioteca importada pode falhar aqui.
- **Cobertura baixa** contribui *fracamente* para **SIM** (feature contestada): menos rede de segurança para pegar regressões, mas a literatura mostra que cobertura sozinha não garante qualidade.

Sinais que puxam para **NÃO** (menos propenso a defeito):

- Complexidade **baixa** + LOC **medio** + churn **baixo** + autores **um/poucos**: perfil de arquivo estável, simples, com dono claro e pouca rotatividade — o retrato do código chato que raramente quebra.
- Cobertura **alta** contribui *fracamente* para **NÃO**.

Combinações ambíguas (onde o modelo tem mais dificuldade): arquivo **longo** e complexo mas com churn **baixo**, autores **um** e cobertura **alta** — grande e intrincado, porém maduro e bem cuidado. É justamente nesses perfis mistos que, em um cenário real onde as features estivessem correlacionadas (Seção 5), a independência ingênua entre elas mais pesaria — os casos de teste da Etapa 4 exploram esse tipo de perfil.

4. Fundamentação Acadêmica

Referências em formato ABNT. Cada entrada destaca o ponto de atenção que ela traz para a modelagem.

McCABE, T. J. A Complexity Measure. *IEEE Transactions on Software Engineering*, v. SE-2, n. 4, p. 308-320, dez. 1976. Define a complexidade ciclomática a partir do grafo de fluxo de controle. O objetivo original é **apoiar o desenho e o dimensionamento de casos de teste** — não prever defeitos. Usar a métrica como preditor de defeito é uma extensão posterior da comunidade, e isso deve ser dito no relatório.

SHEPPERD, M. A Critique of Cyclomatic Complexity as a Software Metric. *Software Engineering Journal*, v. 3, n. 2, p. 30-36, mar. 1988. Mostra que a complexidade ciclomática tem fundamentação teórica frágil e que, para uma classe ampla de software, **não supera a simples contagem de linhas**; em muitos estudos, LOC prevê melhor. Aponta ainda que coeficientes de correlação sobre dados enviesados produzem correlações artificialmente altas. É a base para tratar complexidade × LOC como o par mais problemático do modelo, em código real (Seção 5).

Evidência empírica da relação complexidade × LOC. Estudos posteriores medem uma relação **linear estável** entre complexidade ciclomática e LOC, com coeficiente de determinação alto (R^2 frequentemente acima de 0,90, chegando a $\approx 0,93$ em amostras grandes de código). Ou seja, as duas features carregam quase a mesma informação estatística **em código real** — não nos dados sintéticos deste projeto (Seção 5).

KORU, A. G.; EL EMAM, K.; ZHANG, D.; LIU, H.; MATHEW, D. Theory of Relative Defect Proneness. *Empirical Software Engineering*, v. 13, n. 5, p. 473-498, out. 2008. Resultado central: a relação entre tamanho (LOC) e propensão a defeito segue uma **lei de potência com expoente menor que 1** — módulos menores são **proporcionalmente mais** propensos a defeito. Nota metodológica relevante: os autores **evitaram** calcular "densidade de defeitos" ($\text{bugs} \div \text{LOC}$) porque essa divisão gera correlação negativa artificial com o tamanho; modelaram a probabilidade de defeito diretamente. Nosso rótulo é **binário**, então não corremos esse risco específico — ponto a citar como evidência de rigor na Etapa 4. Consequência para a modelagem: discretizar LOC refletindo a não-linearidade, não assumindo "maior = pior".

NAGAPPAN, N.; BALL, T. Use of Relative Code Churn Measures to Predict System Defect Density. In: *27th International Conference on Software Engineering (ICSE)*, 2005, St. Louis. Proceedings [...]. New York: ACM, 2005. p. 284-292. Mostra que medidas de churn **relativas** (normalizadas pelo tamanho do módulo) predizem densidade de defeito muito melhor que o churn absoluto, e foram desenhadas para **reduzir a dependência de tamanho**.

MATSUMOTO, S.; KAMEI, Y.; MONDEN, A.; MATSUMOTO, K.; NAKAMURA, M. An Analysis of Developer Metrics for Fault Prediction. In: *6th International Conference on Predictive Models in Software Engineering (PROMISE)*, 2010, Timișoara. Proceedings [...]. New York: ACM, 2010. Art. 18. Evidência de que **métricas de desenvolvedor** (quem e quantos mexeram no código) agregam poder preditivo além das métricas de produto. Fundamenta a inclusão de "número de autores distintos".

WEYUKER, E. J.; OSTRAND, T. J.; BELL, R. M. Do Too Many Cooks Spoil the Broth? Using the Number of Developers to Enhance Defect Prediction Models. *Empirical Software Engineering*, v. 13, n. 5, p. 539-559, out. 2008. Investiga diretamente o efeito do número de desenvolvedores sobre a propensão a defeito. Reforça a feature "autores distintos" e dá a intuição de "propriedade do código" usada na Seção 3.4.

INOZEMTSEVA, L.; HOLMES, R. Coverage Is Not Strongly Correlated with Test Suite Effectiveness. In: *36th International Conference on Software Engineering (ICSE)*, 2014, Hyderabad. Proceedings [...]. New York: ACM, 2014. p. 435-445. Estudo controlado: quando se controla o tamanho da suíte de testes, a **cobertura tem correlação fraca** com a capacidade da suíte de detectar defeitos. Base empírica para tratar "cobertura de testes" como feature de sinal duvidoso.

GREN, L.; ANTINYAN, V. On the Relation Between Unit Testing and Code Quality. In: *43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2017, Viena. Proceedings [...]. [S.l.]: IEEE, 2017. p. 52-56. Survey com **235 respostas de sete organizações** mais um estudo de caso em uma delas. Não encontra correlação (ou encontra correlação apenas fraca) entre cobertura de testes unitários e defeitos pós-teste, nem entre percepção de qualidade e prática de testes. Complementa Inozemtseva & Holmes pelo lado da indústria e da percepção dos praticantes.

Síntese dos pontos de atenção para a Etapa 4:

1. Em código real, complexidade ciclomática e LOC compartilham quase toda a informação ($R^2 \approx 0,93$) — a principal violação de independência que a literatura documenta para este domínio (Seção 5). Nos dados sintéticos deste projeto essa correlação foi deliberadamente removida.
2. Tamanho→defeito é **não-linear** (lei de potência, Koru et al.) — "pequeno" não é sinônimo de "seguro". Essa relação é modelada por feature isolada (LOC) e continua presente nos dados sintéticos, pois não depende de correlação com outra feature.
3. Cobertura de testes: relação com defeitos é **contestada** na literatura; feature declarada como limitação assumida.

5. Violações de Independência do Naive Bayes: o que a literatura documenta

O Naive Bayes assume que as features são condicionalmente independentes dado o rótulo. Esta seção é uma discussão **teórica**, não uma antecipação de achado empírico nos dados deste projeto: descreve o que a literatura de engenharia de software empírica documenta sobre a relação entre estas métricas em código **real**.

Par de features	O que a literatura documenta	Referência
Complexidade ciclomática × LOC	Correlação empírica forte e bem estabelecida (R^2 frequentemente acima de 0,90, $\approx 0,93$ em amostras grandes) — as duas métricas carregam quase a mesma informação estatística em código real.	Shepperd (1988); evidência empírica posterior (Seção 4)

Este é o par mais estudado e mais citado como violação de independência nesse domínio, e é por isso que recebe destaque nesta seção. Não há, na literatura consultada para este trabalho, evidência igualmente forte e direta de correlação **entre pares** para as demais quatro features (autores, churn, imports, cobertura) — cada uma delas tem fundamentação própria (Seção 3.2), mas não uma métrica de correlação cruzada tão documentada quanto complexidade × LOC.

Importante — isto não é uma previsão sobre os dados deste projeto. A massa de treinamento sintética da Etapa 2 foi construída, por decisão de projeto ("Naive Bayes puro", ver [CLAUDE.md](#)), com as 6 features **independentes entre si**: nenhuma feature do gerador lê o valor de outra. A matriz de correlação de Pearson calculada sobre esses dados (Etapa 2, §2.2) confirma isso — todos os pares saem com $|r| < 0,10$, incluindo complexidade $\times$ LOC. Ou seja: a violação de independência descrita acima é real na literatura sobre código real, mas **não existe nos nossos dados** — não porque o modelo "escapou" dela, mas porque os dados foram desenhados de propósito para não a conter, isolando assim o comportamento teórico do algoritmo no cenário em que sua premissa central é verdadeira.

A consequência prática dessa escolha — o que ela valida e o que ela deixa de capturar sobre um cenário real de implantação — é o tema central da Reflexão Crítica da Etapa 4, e não é antecipada aqui para não adiantar conteúdo de etapa futura.

6. Nota Metodológica sobre o Uso de IA

O domínio, o conjunto de seis features, os critérios de discretização e as referências deste relatório foram construídos e refinados em **diálogo com uma IA generativa**, usada como parceira de modelagem conforme previsto no enunciado. O processo não foi de aceitação passiva: cada alegação teórica trazida pela IA (a lei de potência de Koru et al., a correlação $R^2 \approx 0,93$ entre complexidade e LOC, a fragilidade da relação cobertura $\rightarrow$ defeitos, a recomendação de churn relativo em Nagappan & Ball) foi submetida a **checagem cruzada contra a literatura original** antes de ser incorporada — verificando autores, ano, veículo de publicação e o que o estudo de fato afirma. Features inicialmente cogitadas (DIT, acoplamento CBO, métricas de Halstead) foram **descartadas** após essa verificação mostrar que correlacionavam fortemente com tamanho, agregando pouca informação nova. Todo o conteúdo aqui foi revisado, adaptado ao domínio escolhido e é passível de defesa oral, linha a linha, pelo autor.

Apêndice A — Perguntas de Defesa (referência rápida)

Por que arquivo e não classe como módulo? Nenhuma das seis features depende do conceito de classe (DIT e CBO foram removidas justamente por isso); arquivo é unidade universal, é a granularidade nativa do Git e evita um parser semântico por linguagem.

Por que só 6 features e não mais? Atende ao mínimo da atividade (6 a 8). Cada feature adicional testada (Halstead, CBO) mostrou forte correlação com LOC, reduzindo a diversidade real de informação sem agregar sinal novo.

Por que os dados são sintéticos e não reais? Exigência explícita da Etapa 2 do enunciado. Pedir dados reais seria descumprir a atividade — não é uma limitação técnica.

Por que as features foram geradas independentes, se a literatura diz que complexidade e LOC se correlacionam no mundo real? Decisão deliberada de simulação ("Naive Bayes puro"): testar o Naive Bayes no cenário em que sua própria suposição central é verdadeira, isolando o comportamento teórico do algoritmo sem misturar com o efeito de uma violação plantada. Isso é declarado explicitamente como limitação de realismo da simulação, não escondido — ver Reflexão Crítica da Etapa 4.

O modelo tem alguma feature fraca? Sim — cobertura de testes tem evidência contestada na literatura quanto à relação com defeitos (Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017). Declarada como limitação assumida, não ignorada.

Se não há violação de independência nos dados, qual é a "crítica" da Etapa 4? A crítica muda de "o modelo erra porque os dados violam a suposição" para "testamos o modelo no cenário ideal onde a suposição é verdadeira; ele se comporta como a teoria prevê, mas isso não reflete um cenário real de implantação, onde essa independência não existiria de fato". A limitação discutida é sobre o **realismo da simulação**, não sobre um erro do modelo.

Apêndice B — Prompts que Moldaram a Modelagem

Registro dos prompts/perguntas-chave dirigidos à IA e do que, na resposta, efetivamente moldou cada decisão. (Documentação exigida pelo enunciado, item "Documente esse diálogo".)

#	Prompt / pergunta à IA (resumo)	O que da resposta foi incorporado (após verificação)
1	"Sugira um domínio de decisão binária para um classificador Naive Bayes que tenha literatura acadêmica sólida, dados fáceis de obter e utilidade prática real."	Escolha de predição de defeitos de software a partir de métricas estáticas entre as opções apresentadas, pela combinação dos três critérios.
2	"Quais métricas de código são preditores clássicos de defeito? Liste com a referência primária de cada uma."	Lista inicial: LOC, complexidade ciclomática, Halstead, DIT, CBO, churn, métricas de desenvolvedor, cobertura. Cada referência foi conferida antes de seguir.
3	"A complexidade ciclomática é redundante com LOC? Existe medida empírica dessa correlação?"	Confirmação via Shepperd (1988) e estudos de relação linear estável (R^2 alto, $\approx 0,93$). Decisão de manter as duas e usar o par como exemplo central de violação de independência, discutido teoricamente na Seção 5.
4	"Módulo maior é sempre mais propenso a defeito? O que diz a literatura sobre a forma funcional dessa relação?"	Koru et al. (2008): lei de potência, expoente < 1 . Decisão de discretizar LOC em risco não-linear (formato U).
5	"Churn absoluto ou relativo? Qual prediz melhor e por quê?"	Nagappan & Ball (2005): churn relativo ao tamanho. Feature 4 redefinida como razão.
6	"Preciso de uma métrica de acoplamento que não dependa de classe. Número de imports serve? Tem risco de virar proxy de tamanho?"	Adoção de nº de imports/dependências via módulo <code>ast</code> ; DIT e CBO removidos por dependerem de classe e correlacionarem com tamanho.
7	"Cobertura de testes prediz bem a ausência de defeitos? Traga evidência contra, se houver."	Inozemtseva & Holmes (2014) e Gren & Antinyan (2017): correlação nula/fraca. Feature 6 mantida, mas declarada como limitação assumida .
8	"Proponha pontos de corte baixo/médio/alto para cada feature, coerentes com código Python real e com as ressalvas acima."	Faixas da Seção 3.3, ajustadas manualmente e marcadas como recalibráveis na Etapa 2.
9	"Revise as referências em ABNT: autor, ano, veículo e o que cada estudo realmente afirma."	Conferência de autores, ano, volume e páginas de cada referência na fonte primária (ex.: Koru et al., <i>Theory of Relative Defect Proneness, Empirical Software Engineering</i> , v. 13, n. 5, 2008; Gren & Antinyan, SEAA 2017, survey de 235 respostas / 7 organizações) e ajuste das paráfrases ao que os textos de fato sustentam.
10	"Ao gerar os dados sintéticos da Etapa 2, devemos plantar as correlações que a literatura documenta (ex.: complexidade $\times$ LOC) ou gerar as 6 features independentes entre si?"	Decisão de projeto "Naive Bayes puro" , registrada no <code>CLAUDE.md</code> : gerar as 6 features independentes entre si, deixando só o rótulo dependente das features. Justificativa aceita: testa o classificador no cenário em que sua própria suposição de independência é verdadeira por construção, isolando o comportamento teórico do algoritmo; a correlação real documentada na literatura passa a ser tratada como limitação de realismo da simulação (Seção 5 e Reflexão Crítica da Etapa 4), não como um achado a "descobrir" nos dados. Essa decisão levou à reescrita do gerador (Etapa 2), da Seção 5 deste relatório e da análise da Etapa 4.

Fim do Relatório de Modelagem — Etapa 1.